

MEMBERSHIP INFERENCE ATTACK

TRUSTWORTHY MACHINE LEARNING

1 INTRODUCTION

In this task, we have to do a Membership Inference Attack on a trained image classification model. My understanding is that we need to find whether a private image sample was used in the training data of the model or not. The model may give more confident output for the images which it already saw during training. So by checking the model output confidence, we can guess if the image is a member or non-member. This is important in Trustworthy Machine Learning because it shows that even if the training data is not directly given, the trained model can still leak some private information.

2 APPROACH

For my approach, I used a confidence-based Membership Inference Attack. The main idea is simple: if the model is very confident for one image, then there is a chance that this image was present in the training set. If the model is less confident, then it may be a non-member image. So I used the maximum softmax probability of the model prediction as the membership score.

First, I loaded the public dataset, private dataset, and trained model from the given files. The public dataset was loaded from pub.pt, the private dataset from priv.pt, and the model from model.pt. I used the same preprocessing as the model training. The images were resized to 32 x 32 and normalized using the given mean and standard deviation.

The model used is ResNet-18, but it is slightly modified for this task. Since the images are small, I changed the first convolution layer to 3 x 3 with stride 1 and padding 1. I also removed the max-pooling layer by replacing it with an identity layer. The final fully connected layer was changed to output 9 classes because this task has 9 image classes.

After loading the model, I passed every image through the model and calculated the softmax probabilities. From these probabilities, I took the maximum value as the confidence score. This score was used as the basic attack score. A higher confidence score means the sample is more likely to be a member.

I also added calibration using the public dataset. If the public dataset contains membership labels, then I train a small calibration model using public confidence scores and their membership labels. This calibration model is only a simple linear layer with one input and one output. It learns to convert the raw confidence score into a better membership probability. This is similar to Platt scaling.

The calibration model was trained using binary cross-entropy loss and Adam optimizer. I used a learning rate of 0.1 and trained it for 200 epochs. The batch size for feature extraction was 64. I kept this method simple because the confidence score itself already gives a useful signal for membership inference. Also, a simple model is easier to reproduce and does not overcomplicate the attack.

The main hyperparameters and choices are:

```
Batch size: 64
Attack score: Maximum softmax confidence
Calibration model: One linear layer
Optimizer: Adam
Learning rate: 0.1
Epochs: 200
Loss: Binary Cross Entropy with Logits
Model: Modified ResNet-18
Number of classes: 9
Image size: 32 x 32
```

I selected this approach because neural networks usually give higher confidence for samples they have already seen during training. So confidence is a direct and useful signal for this attack. Calibration was added to improve the score by learning from the public membership data when available.

3 PUBLIC LEADERBOARD RESULT

For the final submission, I generated a file called submission.csv. This file contains two columns: id and score. The id is the private sample ID, and the score is the predicted membership score. Higher score means the sample is more likely to be a member. My final selected method is:

Confidence-based Membership Inference Attack with public-set calibration

My best public leaderboard result was:

Public leaderboard score: 0.057020

This result was obtained by submitting the generated submission.csv file to the evaluation server. I used the calibrated confidence score because it was simple, stable, and directly matched the idea that training samples usually get higher confidence from the model.

4 CONCLUSION

From this task, I understood that trained machine learning models can leak private information about their training data. Even when the attacker does not see the original training data, the model output can still give clues about whether a sample was used for training or not. This is a serious issue when models are trained on sensitive data like medical images, financial records, or personal user data.

A successful Membership Inference Attack means the model may be memorizing some training samples instead of only learning general patterns. So in real-world systems, we should be careful before releasing model outputs with high confidence values. To make models more trustworthy, we need privacy protection methods like regularization, confidence masking, differential privacy, and proper privacy testing before deployment.

GitHub Repository:https://github.com/Jagadeeswar-reddy-c/TML26_X_01